# ÍNDICES

1. Documentación de reglas modificadas
2. Código no corregido
3. Código corregido

# DOCUMENTACIÓN DE REGLAS MODIFICADAS

## Regla 1

Original: attr-naming-style=snake_case
Modificada: attr-naming-style=UPPER_CASE
Comentarios: Modifica el estilo de nomenclatura de atributos al estilo UPPER_CASE.
Ejemplo positivo: AT1
Ejemplo negativo: at1

## Regla 2

Original: const-naming-style=UPPER_CASE
Modificada: const-naming-style=snake_case
Comentarios: Nomenclatura de constantes siguen el estilo snake_case.
Ejemplo positivo: var
Ejemplo negativo: Var, VAR, vAR

## Regla 3

Original: max-args=5
Modificada: max-args=7
Comentarios: El número máximo de argumentos por función es 7.
Ejemplo positivo: def func1(arg1, arg2, arg3)
Ejemplo negativo: def func2(arg1, arg2, …, arg8)

## Regla 4

Original: max-line-length=100
Modificada: max-line-length=150
Comentarios: La longitud máxima por línea es 150 caracteres.

## Regla 5

Original: single-line-if-stmt=no
Modificada: single-line-if-stmt=yes
Comentarios: Permite, en caso de sentencias if sin else, que su cuerpo se sitúe en la misma línea que la declaración del if.
Ejemplo positivo: if a == 5: b = 3
Ejemplo negativo: (No se proporciona, pero sería el caso con else o con cuerpo en varias líneas)

## Regla 6

Original: argument-rgx=
Modificada: argument-rgx=[a-z_][a-z0-9_]{2,30}$
Comentarios: Exige que los nombres de los argumentos empiecen por una letra minúscula o guion bajo, seguido de un mínimo de 2 y máximo de 30 caracteres que pueden ser letras minúsculas, dígitos o guion bajo.
Ejemplo positivo: def func1(argumento_1)
Ejemplo negativo: def func2(ArGUMENTO2)

## Regla 7

Original: class-attribute-naming-style=any
Modificada: class-attribute-naming-style=snake_case
Comentarios: Exige que la nomenclatura de los atributos de clases siga el estilo snake_case (todo minúsculas).
Ejemplo positivo:

```
class Clase:
def init(self, a):
self.a = a
```

Ejemplo negativo:

```
 class Clase:
def init(self, aA):
        self.aA = aA
        self.A = A
```

## Regla 8

Original: no-docstring-rgx=^_
Modificada: no-docstring-rgx=.*
Comentarios: Habilita que cualquier nombre (función, método, clase) no necesite docstring; no se notifica la falta de docstring en ellos, con excepción del módulo.
Ejemplo positivo 1:

```
        def func1(...):
        """hola"""
```

Ejemplo positivo 2:

```
        def func2(...):
...
```

Ejemplo negativo 1 (excepción): (vacío, se refiere al módulo, que sí requiere docstring)

## Regla 9

Original: docstring-min-length=-1
Modificada: docstring-min-length=3
Comentarios: Para las funciones/módulos que requieren docstring, su longitud mínima es 3 caracteres.
Ejemplo positivo:

```
def func1(...):
"hola"
```

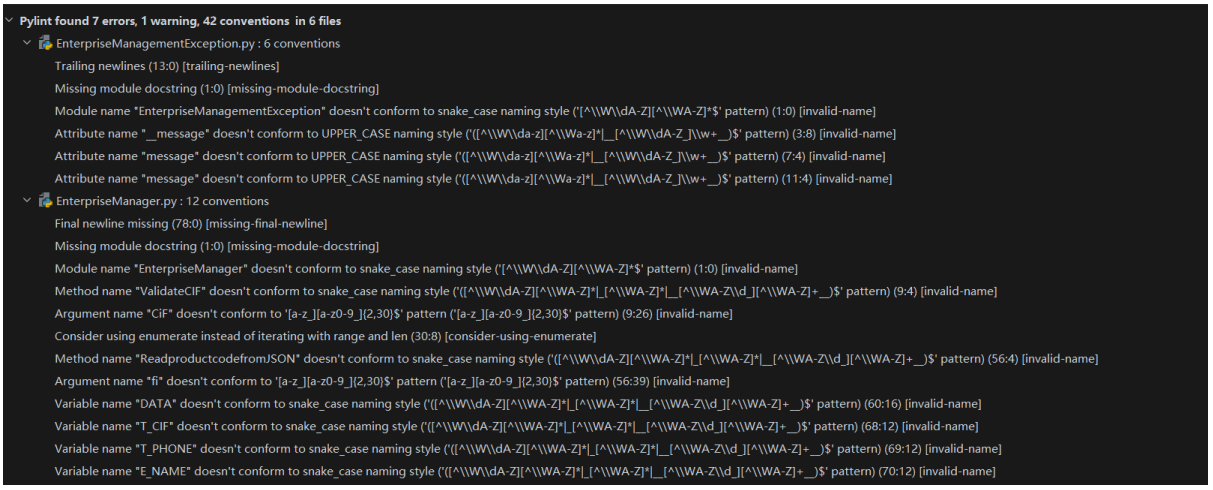
Ejemplo negativo:

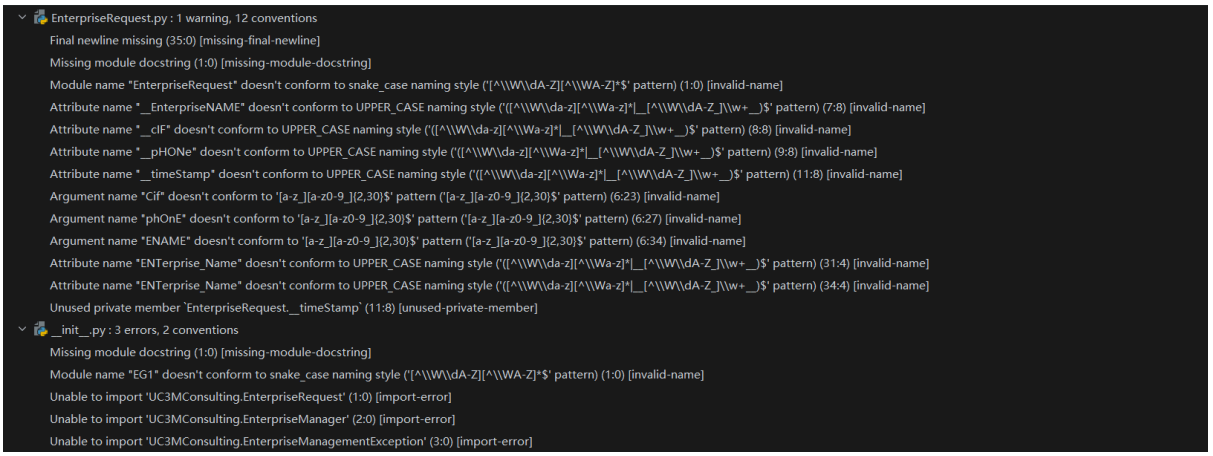
```
def func2(...):
"a”
```

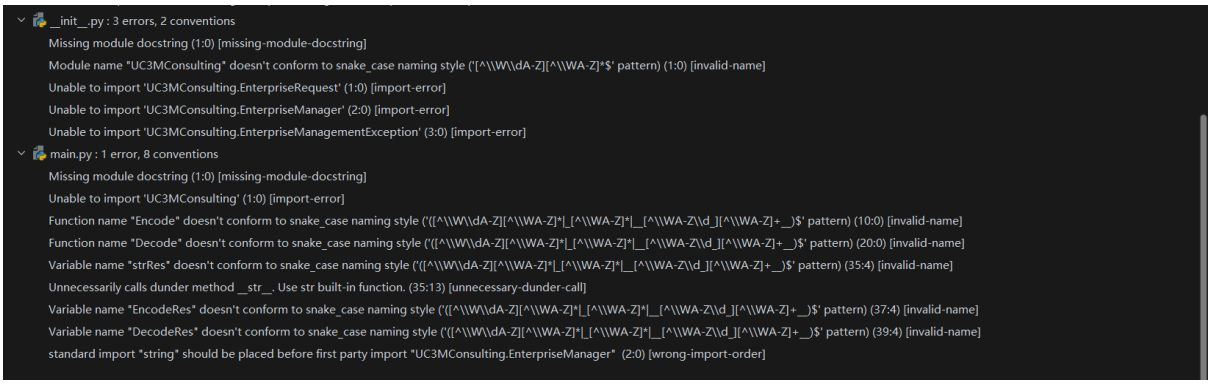
## Regla 10

Original: include-naming-hint=no
Modificada: include-naming-hint=yes
Comentarios: Incluye una sugerencia sobre el formato correcto de los nombres cuando se produce invalid-name.
Ejemplo positivo: Attribute name "message" doesn't conform to UPPER_CASE naming style ('([^\W\da-z][^\Wa-z]*|[^\W\dA-Z_]\w+__)$' pattern) (3:8) [invalid-name]

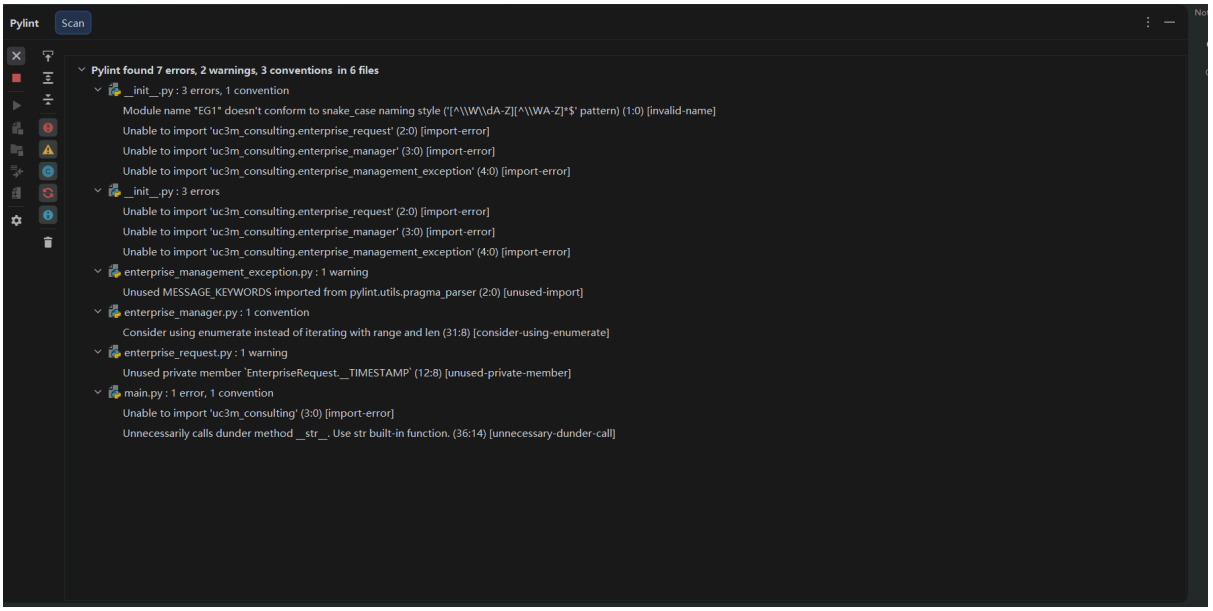
# CÓDIGOS NO CORREGIDOS






# CÓDIGO CORREGIDO



Existen unos avisos, warnings que lo hemos dejados tal cual por siguientes razones:

- La primera línea modifica el formato estándar del nombre del proyecto, este formato está escrito en el enunciado de la práctica, por lo que debe de ser este estilo.
- Import-error, errores de python, no de pylint.
- Funciones, variables y métodos existentes en la base de datos original , que no son utilizados y no contribuyen al funcionamiento del programa, y salen mensajes que avisan que  estos son innecesarios y hemos decididos no eliminarlos.